

Deep Learning 2026 Lab

Rigon Pira

rp57421@ubt-uni.net

University for Business and Technology
Faculty of Computer Science and Engineering
Prishtina, Kosovo

February 2026

1 Introduction

This laboratory project provides practical experience in deep learning, with a focus on Convolutional Neural Networks (CNNs) and the evaluation of machine learning algorithms across two tasks with distinct characteristics.

The first part applies CNN-based classification to the IRIS dataset, where five architectural configurations—varying network depth, activation functions, pooling strategies, and regularization techniques—are systematically compared using the same training and test split. Model interpretability is further analyzed through Leave-One-Out Cross-Validation (LOOCV), confusion matrices, and gradient-based feature importance methods.

The second part addresses a real-world application: the development of an Automatic License Plate Recognition (ALPR) system for Kosovo license plates. This system is implemented using YOLOv8 for object detection and EasyOCR for text recognition. A hybrid dataset is constructed by combining publicly available data from Kaggle with manually annotated Kosovo license plates. Data augmentation techniques are applied extensively, and particular attention is given to correcting data leakage issues during dataset splitting. The model is ultimately evaluated on real-world images captured using a mobile phone camera.

Overall, the project emphasizes systematic experimentation, methodological rigor in evaluation, and quantitative interpretation of results, reinforcing practical skills in classification, object detection, and Python-based data analysis.

2 Software Environment

Both experiments were carried out in Python 3.10.9 on a local machine (laptop) equipped with an Intel Core i5-1135G7 processor and 16 GB of RAM, running Windows 11 Home (x64).

The IRIS experiments were executed entirely on CPU using TensorFlow/Keras 2.20.0 [1], NumPy 2.2.6, Pandas 2.3.3, and Scikit-learn 1.7.2 [2] (for stratification, LOOCV, evaluation metrics, and PCA/t-SNE), with visualizations produced via Matplotlib 3.10.8 and Seaborn 0.13.2.

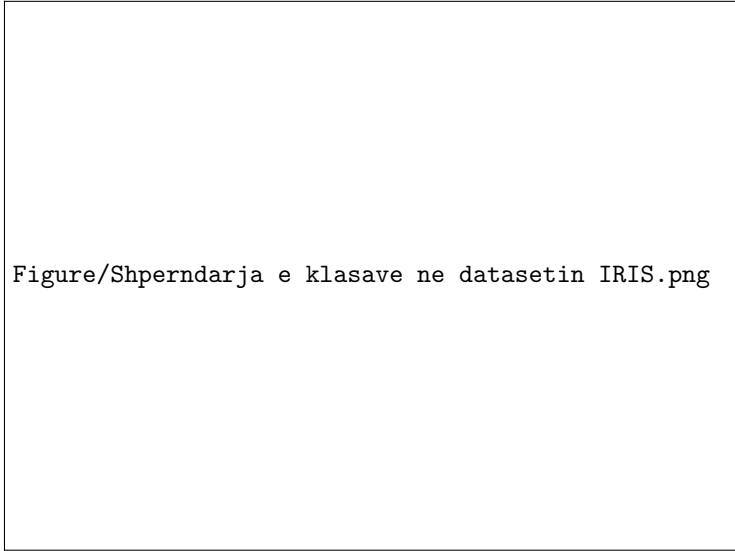
The license plate recognition experiments were built on PyTorch 2.10.0 [3] and Ultralytics 8.4.14 for YOLOv8 [4], along with EasyOCR 1.7 [5], Albumentations 1.3 [6], and OpenCV 4.8. Due to computational requirements, model training was migrated to Google Colab using an NVIDIA Tesla T4 GPU, while inference and the OCR pipeline were tested on the local CPU.

Execution Time: The total wall-clock time for the IRIS experiments was approximately 14–15 minutes on CPU, with LOOCV taking around 14.5 minutes and the architectural comparison requiring about 1 minute. For the license plate task, data augmentation and dataset splitting required approximately 12–17 minutes on CPU. Each YOLOv8 experiment took approximately 25–45 minutes on the T4 GPU, while full OCR-based evaluation over 1,164 images required approximately 10–15 minutes on CPU.

3 Convolutional Neural Network Classification on the IRIS Dataset

3.1 Dataset Description

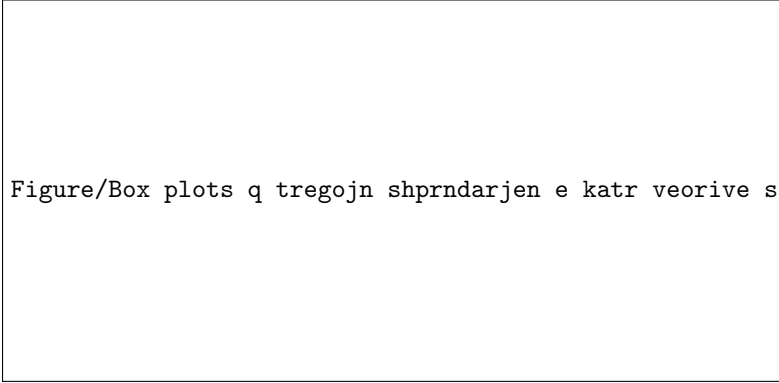
The IRIS dataset, introduced by R. A. Fisher [7] and included in the UCI Machine Learning Repository, is a canonical benchmark for supervised classification. It comprises 150 samples evenly distributed across three species (50 samples each): *Iris setosa*, *Iris versicolor*, and *Iris virginica* (Figure 1). Each sample is described by four numerical attributes measured in centimetres: sepal length, sepal width, petal length, and petal width.



Figure/Shperndarja e klasave ne datasetin IRIS.png

Figure 1: Balanced class distribution of the IRIS dataset.

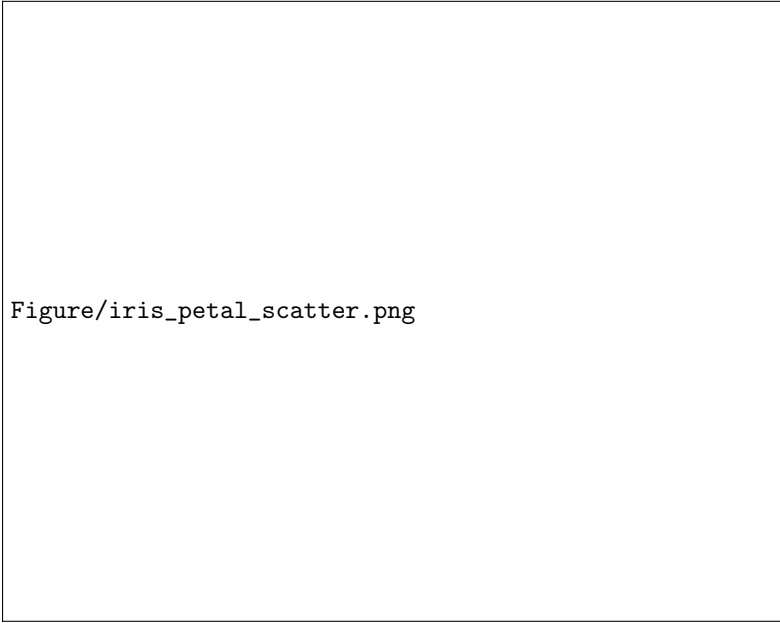
Box plots of the per-feature distributions across species (Figure 2) reveal substantial between-class variability, particularly in the petal attributes: *Iris setosa* exhibits the smallest values whereas *Iris virginica* the largest, suggesting that petal features are more discriminative than sepal features.



Figure/Box plots q tregojn shprndarjen e katr veorive sipas specieve..png

Figure 2: Distribution of feature values across Iris species using box plots.

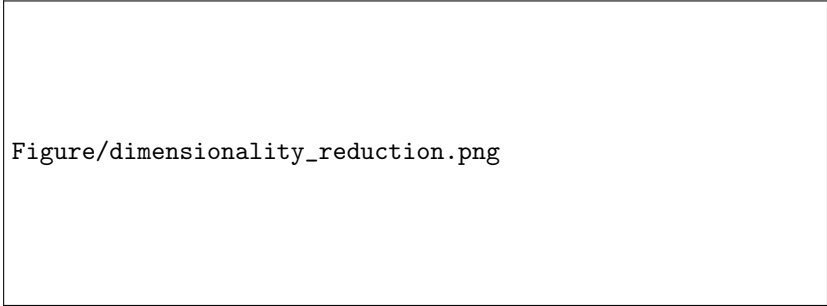
The two-dimensional exploratory analysis on the petal features (Figure 3) confirms that *Iris setosa* is linearly separable, while *Iris versicolor* and *Iris virginica* overlap substantially. This overlapping region constitutes the principal challenge of the classification task.



Figure/iris_petal_scatter.png

Figure 3: Class separation in the two-dimensional petal feature space.

Dimensionality reduction using Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) (Figure 4) corroborates this structure. The first two principal components capture most of the dataset variance and clearly isolate *Iris setosa*, while t-SNE provides a more detailed visualization of the overlap between *Iris versicolor* and *Iris virginica*.



Figure/dimensionality_reduction.png

Figure 4: Visualization of the dataset in two dimensions using PCA and t-SNE.

The low dimensionality, balanced class distribution, and limited noise level make the IRIS dataset well suited for controlled experiments. Under these conditions, differences in model performance can be attributed primarily to architectural design and hyperparameter choices. Although the data are not

image-based, the IRIS dataset was deliberately selected to investigate the suitability of Convolutional Neural Networks (CNNs) for tabular numerical data.

3.2 Methodology

3.2.1 Leave-One-Out Cross-Validation (LOOCV)

To obtain an unbiased performance estimate on a small dataset, Leave-One-Out Cross-Validation was employed: at each iteration one sample is held out for testing while the remaining 149 are used for training. On IRIS ($n = 150$), the model is therefore retrained 150 times, ensuring that every sample serves exactly once as a test instance. This procedure reduces estimator variance and provides a faithful picture of the model’s generalization ability, at a high but tractable computational cost at this scale. The final performance is reported as the mean across the 150 folds.

3.2.2 Data Preprocessing

The dataset was loaded directly from the `iris.data` file provided on the course Moodle page, which contains nine missing entries encoded either as the string `NaN` or as the character `?`. These tokens were treated as missing values during parsing and replaced with the column-wise median computed exclusively from `iris.data` itself; median imputation was preferred over listwise deletion as it is robust to outliers and preserves both the full sample size (150) and the original class balance of 50 samples per species, which is required for the Leave-One-Out Cross-Validation protocol described in Section 3.2.1.

The four real-valued features were then standardized via z-score normalization, yielding zero mean and unit variance, and reshaped from $(150, 4)$ to $(150, 4, 1)$ to enable Conv1D operations over the four features treated as a one-dimensional sequence. Finally, the categorical labels were one-hot encoded into three columns for use with the categorical cross-entropy loss.

No external dataset is referenced at any stage of the pipeline.

3.2.3 CNN Architectures

Five configurations were implemented to investigate the impact of architectural choices (Table 1). All models were compiled with the Adam optimizer ($\text{lr}=0.001$), categorical cross-entropy loss, and accuracy as the evaluation metric.

Table 1: CNN architecture specifications and parameter counts.

Configuration	Conv Layers	Activation	Key Features	Parameters
Baseline CNN	1	ReLU	MaxPooling, Dropout (0.3)	4,451
Deep CNN	3	ReLU	BatchNorm, MaxPooling, Dropout (0.2-0.4)	67,075
No MaxPooling	2	ReLU	BatchNorm, No Pooling	91,587
Tanh Activation	2	Tanh	BatchNorm, MaxPooling	50,243
L2 Regularization	1	ReLU	BatchNorm, L2 ($\lambda = 0.01$)	25,411

The Baseline CNN consists of a single Conv1D layer (32 filters, kernel size 2), MaxPooling, and fully connected layers. The Deep CNN extends this with three convolutional blocks and BatchNormalization. The No MaxPooling variant probes the role of spatial downsampling, the Tanh configuration examines an alternative activation function, and the L2 Regularization model applies a weight penalty ($\lambda = 0.01$).

The present study deliberately limits hyperparameter variation to network depth, activation, pooling, and regularization. Filter size, padding, and stride were held fixed (kernel=2, padding="valid", stride=1) due to the structural constraints of the input: with only four numerical features reshaped to (4, 1), kernel sizes greater than 2 would span the entire feature vector (collapsing the convolution into a fully connected operation), strides greater than 1 would aggressively downsample an already low-dimensional input, and padding variations carry limited semantic meaning. Consequently, the retained hyperparameters are precisely those that admit a meaningful range of values for low-dimensional tabular data.

3.2.4 Training Configuration

The dataset was partitioned with an 80–20 stratified split (120 train / 30 test). Training employed the Adam optimizer (lr= 0.001), categorical cross-entropy loss, batch size 16, a maximum of 100 epochs, early stopping with patience=15 on validation loss (restoring best-epoch weights), and ReduceLROnPlateau (factor=0.5, patience=5, min LR= 10^{-7}). LOOCV was further applied to the best-performing model in order to obtain an unbiased estimate of generalization on $n = 150$.

3.2.5 Feature Importance Analysis

Gradient-based feature importance was computed using TensorFlow’s `GradientTape`: for each feature, the importance is defined as the mean absolute gradient of the model output with respect to the input across all samples. Scores were normalized to sum to unity, enabling direct comparison of relative contributions and identifying the input dimensions most strongly influencing the classification.

3.3 Results

3.3.1 Model Performance Comparison

Three configurations (Baseline, Deep, L2) attained 100% test accuracy, the No MaxPooling variant 96.67%, while the Tanh model trailed at 86.67%, a 13.33 percentage-point gap that illustrates how decisive the choice of activation function remains, even on a low-dimensional dataset (Table 2).

The Tanh configuration restored its best weights from epoch 3 and was early-stopped at epoch 18: although training accuracy continued to improve toward 95–97%, the validation accuracy plateaued near 66.67% with the validation loss diverging from the training loss after epoch 5. This pattern is the well-known

signature of the vanishing-gradient problem combined with overfitting on a small dataset, and it explains both the lower final accuracy and the inability of further training to recover.

In contrast, the three top-performing ReLU-based models exhibited well-aligned training and validation curves throughout, and the No MaxPooling variant, despite removing the spatial-downsampling step still produced a competitive accuracy, indicating that pooling contributes incrementally rather than fundamentally on this task.

Table 2: Model performance on the 30-sample test set.

Configuration	Test Acc. (%)	Epochs	Final Val. Loss
Baseline CNN	100.00	100	0.0425
Deep CNN	100.00	94	0.0049
L2 Regularization	100.00	100	0.1493
No MaxPooling	96.67	59	0.0944
Tanh Activation	86.67	3 (early stop)	0.5370



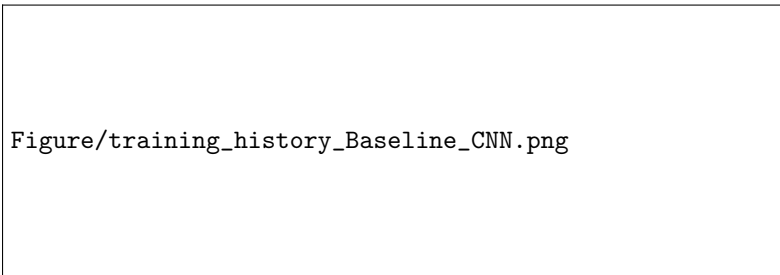
Figure 5: Multi-panel visualization showing accuracy bars, training curves, and confusion matrices for all configurations.

Figure 5 provides a comprehensive comparison: the bars show relative accuracy, the middle panels the evolution of training and validation accuracy, and the lower panels the confusion matrices, confirming perfect classification for the three top-performing models.

3.3.2 Training Dynamics

The Baseline CNN (Figure 6) exhibited rapid learning during the first 30 epochs (90% accuracy by epoch 17), followed by gradual refinement up to epoch 100. The learning-rate-reduction mechanism was triggered at epoch 91 ($0.001 \rightarrow$

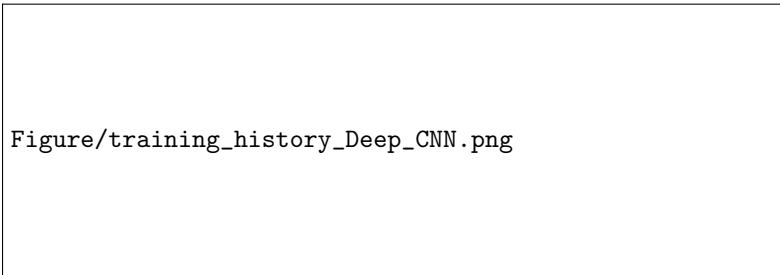
0.0005). The train-validation gap remained below 1%, ruling out overfitting.



Figure/training_history_Baseline_CNN.png

Figure 6: Dual-axis plot showing the evolution of training/validation accuracy and loss across 100 epochs (Baseline CNN)..

The Deep CNN (Figure 7) showed stable convergence in the long run, with minimum validation loss (0.0049) at epoch 94, suggesting that BatchNorm and additional depth jointly contribute to training stability.



Figure/training_history_Deep_CNN.png

Figure 7: Training dynamics of the Deep CNN, exhibiting the most stable convergence.

Figures 8 and 9 show good convergence for the alternative configurations: No MaxPooling reaches 96.67%, while L2 Regularization attains 100% but requires the full 100 epochs, the weight penalty slows learning while improving generalization.

Figure/training_history_CNN_without_MaxPooling.png

Figure 8: Training dynamics of the CNN without MaxPooling.

Figure/training_history_CNN_with_L2_Regularization.png

Figure 9: CNN with L2 Regularization.

3.3.3 Leave-One-Out Cross-Validation Performance

LOOCV applied to the Baseline CNN yielded **95.33% accuracy** (143/150), a more conservative yet realistic estimate than that obtained on the held-out test set. Six of the 7 errors occurred between *versicolor* and *virginica* (3 in each direction), reflecting the natural phenotypic overlap, while a single *Iris setosa* sample (index 41) was misclassified as *versicolor* (Table 3, Figure 10). The model is therefore well balanced, exhibiting no systematic bias.

Table 3: LOOCV classification metrics across 150 samples.

Species	Precision	Recall	F1	Support
Iris setosa	1.00	0.98	0.99	50
Iris versicolor	0.92	0.94	0.93	50
Iris virginica	0.94	0.94	0.94	50
Macro Average	0.95	0.95	0.95	150

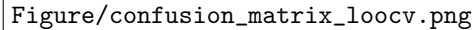


Figure 10: 3×3 confusion matrix showing near-perfect *setosa* classification (49/50) and symmetric *versicolor*–*virginica* confusion (3+3)

3.3.4 Misclassification Analysis

Inspection of the 7 misclassified samples (indices 41, 70, 77, 83, 106, 119, 133), four of which are illustrated in Figure 11, shows that they lie at the boundary of the feature space. The six *versicolor*–*virginica* errors exhibit a familiar pattern: the *versicolor* samples classified as *virginica* had petal measurements at the upper end of the *versicolor* distribution (close to *virginica*), while *virginica* samples classified as *versicolor* exhibited petal dimensions at the lower end. This pattern is consistent with botanical taxonomy: *I. versicolor* and *I. virginica* share a more recent evolutionary ancestor than either does with *I. setosa*.

The single *setosa* misclassification (sample 41) has a different origin. Its raw entry in `iris.data` was 4.5, 2.3, 1.3, ?, *Iris-setosa*, atypical sepal dimensions (sepal length 4.5 cm, sepal width 2.3 cm, both at the low end of *setosa*) combined with an originally missing petal-width value. After median imputation (Section 3.2.2) the petal width was set to 1.3 cm, which falls within the *versicolor* distribution (typical *versicolor* petal width: 1.0–1.8 cm); the resulting feature vector therefore lay closer to the *versicolor* cluster than to *setosa*, producing this isolated boundary error. The case illustrates a known trade-off of global imputation: replacing a missing value by a class-agnostic statistic can occasionally place an atypical sample on the wrong side of the decision boundary, although in this dataset the effect is limited to a single sample.

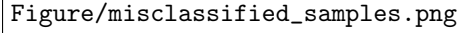
The figure is a placeholder for an analysis of LOOCV misclassified samples.

Figure 11: Analysis of LOOCV misclassified samples.

3.3.5 Feature Importance Rankings

Gradient-based importance (Table 4), visualized in Figure 12) confirms the dominance of the petal features: petal width (50.43%) + petal length (34.69%) together account for 85.12% of the decision attribution. The sepal features contribute only marginally (sepal width 9.95%, sepal length 4.93%), in agreement with botanical knowledge that petal morphology exhibits greater interspecies variation.

Table 4: Gradient-based feature importance rankings.

Feature	Importance	Contribution (%)
Petal Width	0.5043	50.43
Petal Length	0.3469	34.69
Sepal Width	0.0995	9.95
Sepal Length	0.0493	4.93

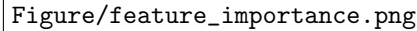
The figure is a placeholder for CNN-learned feature importance.

Figure 12: CNN-learned feature importance.

3.3.6 Feature Correlation Analysis

The correlation matrix (Figure 13) reveals a strong positive correlation between petal length and petal width (Pearson $r = 0.94$), a moderate positive correlation

between sepal length and the petal features ($r \approx 0.83$), and a weak negative correlation between sepal width and the remaining features ($r \approx -0.30$). The high petal length–width correlation explains their joint importance as redundant yet complementary information, sepal width, with its negative correlation, supplies orthogonal information that is useful for boundary cases.

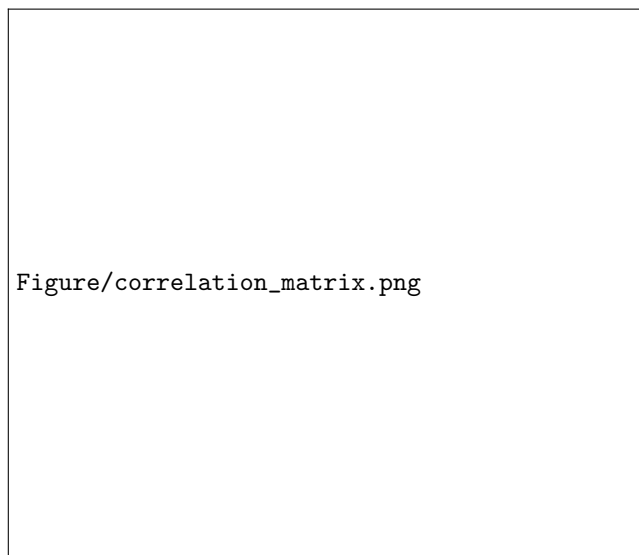


Figure 13: Pearson correlation coefficients between features.

3.3.7 Model Internal Representations

The learned convolutional filter weights (Figure 14) are consistent with the quantitative importance analysis (Section 3.3.5), which attributes 85.12% of the decision to the petal features. While individual filter patterns vary across runs due to random initialization, the convergence between gradient-based attribution, correlation analysis, and the model’s learned representations supports the conclusion that the network has learned a botanically meaningful feature hierarchy.



Figure 14: Convolutional filter weights learned by the model.

3.4 Discussion

Hyperparameter impact Architectural choices have a perceptible effect: activation is the critical factor (ReLU 100% vs Tanh 86.67% — Tanh suffers from the vanishing-gradient problem), MaxPooling contributes to abstraction and invariance (its removal costs 3.33 percentage points), depth shows diminishing returns on this low-dimensional dataset (Baseline 4,451 is on par with Deep 67,075 parameters in accuracy), and L2 ($\lambda = 0.01$) prevents overfitting without compromising generalization.

Feature selection Three converging methods identify the petal features as optimal: (i) the high correlation ($r = 0.94$) indicates redundant yet complementary information, (ii) gradient-based importance attributes 85.12% of the decisions to the petals, and (iii) PCA, dominated by petal variance, captures most of the data spread. The convergence is consistent with botanical taxonomy, petal morphology evolves more rapidly than sepal morphology.

Learned representations The internal representations are interpretable through three lenses: (i) the quantitative importance scores (petal width 0.5043, etc.), (ii) the first-layer filter weights, which focus on the petal dimensions, and (iii) the error pattern, confined to the versicolor–virginica overlap region.

Generalization and limitations. LOOCV (95.33%) provides a more conservative estimate than the held-out test set (100%), reflecting the true generalization gap. The symmetric 3+3 versicolor–virginica error pattern, complemented

by a single setosa boundary case caused by median imputation, indicates balanced uncertainty rather than systematic bias. Principal limitations: (i) the small dataset size ($n = 150$) limits the statistical power to discriminate among the top-performing models, (ii) CNNs are designed for data with spatial locality; for tabular data without spatial structure, MLPs or tree-based models may achieve comparable performance with reduced complexity, (iii) the study used the raw measurements without domain-specific feature engineering (e.g. petal aspect ratio), (iv) LOOCV becomes prohibitive for larger datasets, and (v) a complete visualization of the layer activations was not produced due to technical constraints.

3.5 Conclusion

The study successfully applied CNNs to the IRIS dataset, achieving 100% test accuracy and 95.33% LOOCV accuracy with the Baseline CNN. The research questions were addressed as follows: (1) hyperparameters have a substantial effect, activation is the most critical factor (100% vs 86.67%), and depth exhibits diminishing returns on small datasets, (2) the discriminative features are identified through the convergence of correlation analysis, gradient-based attribution, and PCA, the petal features dominate with 85.12%, and (3) the learned representations are interpretable and consistent with botanical knowledge, with six of the seven errors concentrated at the versicolor–virginica boundary.

The principal findings are that simple architectures (4,451 parameters) match the performance of more complex models (67,075), suggesting that model complexity should be matched to the data, that LOOCV provides a robust performance estimate, and that the feature hierarchy learned by the CNN reflects expert botanical knowledge, demonstrating interpretability despite the model’s reputation as a black box. Future work could investigate attention mechanisms for further interpretability, ensemble methods for boundary cases, and comparisons with classical methods to assess whether CNN complexity is justified for low-dimensional tabular data.

4 License Plate Detection and Recognition

4.1 Problem Specification and Background

Automatic License Plate Recognition is a central task in parking management, tolling systems, stolen-vehicle identification, and access control to restricted areas. Recent advances in deep learning, in particular single-stage detectors of the YOLO family [8], have made deployment feasible even on resource-constrained devices. Detection in real-world conditions, however, contends with surface contamination, mechanical deformations, unfavourable illumination, non-frontal viewpoints, low resolution, and motion blur, factors that render automatic licence plate recognition a non-trivial task.

Specifics of Kosovar plates: License plates in the Republic of Kosovo follow a structured format:

RKS XX–NNN–LL

Where:

- **RKS** is the country prefix (Republic of Kosovo), displayed on a blue strip on the left side.
- **XX** is the 2-digit regional code (01 Pristina, 02 Mitrovica, 03 Peja, 04 Prizren, 05 Ferizaj, 06 Gjilan).
- **NNN** represents a 3-digit vehicle identifier.
- **LL** represents a 2-letter vehicle identifier.

4.2 Dataset Description

The dataset combines two sources:

- **Kaggle** [9] (“Car License Plate Detection” by Larxel) – 433 international images (Pascal VOC XML format), converted to YOLO format using `step0_prepare_kaggle.py`.
- **Kosovar plates** – 270 images (45 per region across 6 regions), manually annotated using `LabelImg`. Representative samples covering the six regions are shown in Figure 15.



Figure/License Plate Recognition/fig01_dataset_samples.png

Figure 15: Sample Kosovar plates from the six regions of Kosovo.

4.2.1 Data Augmentation

Asymmetric augmentation was applied via Albumentations [6]: $\times 15$ for the Kosovar subset ($\sim 4,050$ additional images) and $\times 5$ for the Kaggle subset ($\sim 2,165$ additional), thereby weighting the domain-specific source more heavily. After augmentation the dataset contains $\sim 6,900$ images. The pipeline encompasses four families of transformations:

- **Geometric:** HorizontalFlip, Affine (scale, translate, rotation $\pm 12^\circ$, shear), Perspective transform.
- **Photometric:** RandomBrightnessContrast, CLAHE, HueSaturationValue, RandomGamma.
- **Noise and blur:** GaussNoise, ISONoise, MotionBlur, GaussianBlur.
- **Environmental:** RandomRain, RandomShadow, RandomSunFlare, RandomFog.

A representative example of the augmentation output, the original image alongside five generated variants is shown in Figure 16, illustrating the diversity introduced by the combined geometric, photometric, and environmental transformations.



Figure/License Plate Recognition/fig02_augmentation_demo.png

Figure 16: Augmentation overview: original image and 5 variants generated by the Albumentations pipeline.

4.2.2 Train/Val/Test Split without Data Leakage

The 70/15/15 partition is applied at the level of the base image, not at the level of the augmented file: all augmentations of a given base stem (e.g., 01_PR-01) are assigned to the same split, with the Kaggle and Kosovar subsets partitioned separately for balance. In an earlier version, shuffling directly over augmented files yielded mAP@50 ~99% that did not generalize - a clear case of unfair evaluation. The fix brought the metrics into a realistic range. The procedure is implemented in Listing 1: a regular expression strips the `_orig/_augN` suffix to recover the base stem, files are grouped by (source, base_stem) prior to shuffling, and the split is performed at the stem level so that all augmentations of a given image remain in the same partition. The resulting partition is summarized in Table 5.

Listing 1: Base-stem-level dataset partitioning.

```
AUG_SUFFIX_RE = re.compile(r'_(orig|aug\d+)$')

def base_stem_of(filename):
    """Strip the _orig / _augN suffix to recover the base
    stem."""
    stem = Path(filename).stem
    match = AUG_SUFFIX_RE.search(stem)
    return stem[:match.start()] if match else stem

# Group by (source, base_stem) BEFORE shuffling
groups = defaultdict(lambda: defaultdict(list))
for img_file in os.listdir(all_img_dir):
```

```

source = KAGGLE_SOURCE_KEY if img_file.startswith('KG_')
    else img_file[:2]
base = base_stem_of(img_file)
groups[source][base].append(img_file)

# Shuffle is performed at the BASE-STEM level, not at the
augmented-file level
for source, regional_groups in groups.items():
    base_stems = sorted(regional_groups.keys())
    random.shuffle(base_stems)
    n = len(base_stems)
    splits = {
        'train': base_stems[:int(n * TRAIN_RATIO)],
        'val': base_stems[int(n * TRAIN_RATIO):int(n * (
            TRAIN_RATIO + VAL_RATIO))],
        'test': base_stems[int(n * (TRAIN_RATIO + VAL_RATIO
            )):]
    }

```

Table 5: Dataset partitioning after the data-leakage correction.

Split	Files	Percentage
Train	4,794	70%
Val	960	15%
Test	1,164	15%
TOTAL	6,918	100%

4.2.3 Architectural decision: single-class detection + OCR for the region

A key methodological choice is to use a YOLO detector with a single class `license_plate`, and to recover the regional code (01–06) from OCR over the recognized text. The alternative of training a 6-class detector would force the model to learn visual context specific to each region (background, typical illumination, vehicle types), which would harm generalization to plates from new scenes. By formulating the problem as single-class detection, the model learns the universal representation of a license plate (a high-contrast rectangle of text in a typical image position), while regional identification is delegated to the OCR. The benefits are: (i) better generalization to regions with few training images, (ii) auditability: errors are cleanly partitioned into detection vs OCR, and (iii) scalability: adding a new region requires no retraining. The validity of this design is confirmed empirically in Section 4.5 by 100% accuracy on real images.

4.3 Methodology

4.3.1 YOLOv8 Architecture

The detector employed is YOLOv8 [4] (Ultralytics), a state-of-the-art single-shot architecture that performs detection and classification in a single forward pass. The topology (Figure 17) consists of three components:

- **Backbone (CSPDarknet)** – extracts hierarchical features P1–P5 with Conv + C2f blocks.
- **Neck (FPN/PAN)** – fuses multi-scale features via upsampling and additional C2f blocks for multi-scale detection.
- **Head** – three detection heads with CIoU + DFL losses (regression) and BCE loss (classification).

Figure/License Plate Recognition/fig00_yolov8_topology.png

Figure 17: YOLOv8 topology: Backbone (CSPDarknet), Neck (FPN/PAN), and Head (3 detection layers).

The **YOLOv8n** variant has 130 layers, 3,011,043 parameters (3.0M) and 8.2 GFLOPs. For our task, only the Head was modified: from the 80 COCO classes to 1 class (`license_plate`), yielding a final parameter count of **3,005,843**. The Backbone and Neck were inherited unchanged (transfer learning from COCO).

4.3.2 Comparative experiments

Two model variants were compared at identical resolution (640×640), varying only the model scale (Table 6). Holding the remaining parameters constant ensures a controlled comparison.

Table 6: Configuration of the two comparative experiments.

Experiment	Model	Params	Resolution	Batch	Max Epochs
exp1_v8n_640	YOLOv8n	3.0M	640×640	32	80
exp2_v8s_640	YOLOv8s	11.2M	640×640	16	80

A third experiment at 832×832 was interrupted at epoch 62/80 due to Colab GPU quota constraints. The decision not to retrain was justified by (i) the methodological cleanliness of comparing a single variable at a time and (ii) the 100% accuracy of the winning model on real images, which renders any further improvement marginal.

4.3.3 Training Configuration

Both experiments were trained using the following configuration:

- **Optimizer:** AdamW
- **Initial Learning Rate:** 0.001
- **Final Learning Rate Factor:** 0.01 (cosine annealing)
- **Weight Decay:** 0.0005
- **Cosine LR Schedule:** Yes (cos_lr=True)
- **Loss Function:** CIoU + DFL (regression) + BCE (classification)
- **Maximum Epochs:** 80
- **Patience (Early Stopping):** 15 epochs
- **Save Period:** Every 20 epochs

YOLO-specific augmentation parameters (applied in addition to Albumentations used in Step 1):

- `degrees` = 5.0
- `translate` = 0.05
- `scale` = 0.1
- `fliplr` = 0.0 (horizontal flip handled in Albumentations)
- `mosaic` = 1.0 (YOLO-specific: combines 4 images)
- `close_mosaic` = 10 (mosaic disabled in the final 10 epochs)
- `hsv_h` = 0.015, `hsv_s` = 0.3, `hsv_v` = 0.2

Hardware: Training was performed on an NVIDIA Tesla T4 GPU using Google Colab. **Training Time:** Approximately 25–45 minutes per experiment.

4.3.4 OCR Pipeline

YOLO produces only the bounding boxes, text extraction is performed by EasyOCR [5] with additional pre- and post-processing.

Phase 1 — Preprocessing with 6 variants: (1) Gray zone (grayscale of the central region), (2) CLAHE, (3) Denoised (Non-local Means), (4) Threshold (Otsu or adaptive), (5) Sharpened (3×3 after denoising), and (6) Letters Zoom ($\times 3$ magnification of the rightmost 30% to better read the LL pair).

Phase 2 — Dual-allowlist OCR: EasyOCR is run twice on each variant, once restricted to digits (which removes the 7-vs-T and 0-vs-O ambiguities) and once restricted to letters (which avoids the H-vs-Q and V-vs-D ambiguities).

Phase 3 — Weighted voting: The region is determined via a four-stage cascade, the three middle digits are then selected by weighted voting, the core of which is given in Listing 3. Each candidate receives a $0.25\times$ penalty when it begins with the regional code (mitigating contamination from the prefix), an additional reduction when the candidate equals the region itself, and a $1.4\times$ boost for fragments located in the central image zone. The final two letters are extracted with priority given to the `letters_zoom` variant.

Phase 4 — Deduplication: An IoU threshold of 0.5 is used to remove overlapping boxes that survive the standard NMS step, the implementation is shown in Listing 2, which retains only the highest-confidence box within each overlapping cluster.

Listing 2: Manual bounding-box deduplication.

```
def deduplicate_boxes(boxes, iou_threshold=0.5):
    """Keep only the highest-confidence box in each
    overlapping cluster."""
    if boxes is None or len(boxes) <= 1:
        return list(boxes) if boxes is not None else []
    boxes_list = sorted(list(boxes), key=lambda b: -float(b.
        conf.item()))
    keep = []
    for box in boxes_list:
        coords = box.xyxy[0].tolist()
        is_dup = any(_bbox_iou(coords, k.xyxy[0].tolist()) >
            iou_threshold
            for k in keep)
        if not is_dup:
            keep.append(box)
    return keep
```

Listing 3: Weighted voting for the three middle digits.

```
# Weighted voting for the three middle digits
for cand, frag_w in cand_from_frag:
    weight = base_conf * frag_w
    if region_final and cand[:2] == region_final:
        weight *= 0.25          # penalty: regional prefix
```

```

    if region_final and cand == region_final:
        weight *= 0.1          # penalty: candidate equals the
                                region
    if mid_lo <= x_pos <= mid_hi:
        weight *= 1.4          # boost: central zone
    digit_candidates.append((cand, weight, x_pos))

cand_weights = defaultdict(float)
for cand, w, _ in digit_candidates:
    cand_weights[cand] += w
digits_found = max(cand_weights.items(), key=lambda kv: kv
[1])[0]

```

Final output format: RKS XX-NNN-LL.

4.4 Detection Results

4.4.1 Comparison of experiments

The smaller model `exp1_v8n_640` outperformed the v8s variant despite having 3.7x fewer parameters (Table 7).

Table 7: Performance of the two experiments on the validation set.

Experiment	Model	Best Epoch	mAP@50	mAP@50-95	Precision	Recall	Total Epochs
exp1_v8n_640	YOLOv8n (3.0M)	59	0.9477	0.6144	0.9573	0.9312	80 (full)
exp2_v8s_640	YOLOv8s (11.2M)	40	0.9419	0.5913	0.9396	0.9392	70 (early stop)

4.4.2 Interpretation: bias–variance tradeoff

The observation that v8n outperforms v8s is consistent with the classical bias–variance tradeoff: (i) the higher capacity of v8s is not fully exploited on $\sim 6,900$ images, making it more susceptible to overfitting on training-specific features, (ii) v8s peaked at epoch 40 and was halted by early stopping at epoch 70, whereas v8n continued to improve until epoch 59 along a smoother trajectory, and (iii) the lower-capacity model has higher bias but lower variance, which is favourable when the data are modest. The methodological takeaway is that capacity should be calibrated to the effective complexity of the data.

4.4.3 Training curves

The training dynamics of the two experiments are shown in Figures 18 and 19, which display the evolution of the box, classification, and DFL losses alongside the precision, recall, and mAP metrics on the validation set. The `exp1_v8n_640` run (Figure 18) exhibits a smooth trajectory with sustained improvement up to epoch 59, whereas `exp2_v8s_640` (Figure 19) converges more rapidly but plateaus after epoch 40, triggering early stopping at epoch 70.



Figure/License Plate Recognition/fig03_training_curves_exp1_v8n_640.png

Figure 18: Training curves for exp1_v8n_640.



Figure/License Plate Recognition/fig04_training_curves_exp2_v8s_640.png

Figure 19: Training curves for exp2_v8s_640: faster convergence but plateau after epoch 40.

Three principal behaviours emerge from these curves:

- **No overfitting:** the train-validation gap remains narrow.
- **Stable convergence:** losses decrease monotonically and metrics rise gradually.

- **Early stopping:** not triggered for exp1 (improvements continued through epoch 59), but triggered for exp2 (plateau after epoch 40, halted at epoch 70).

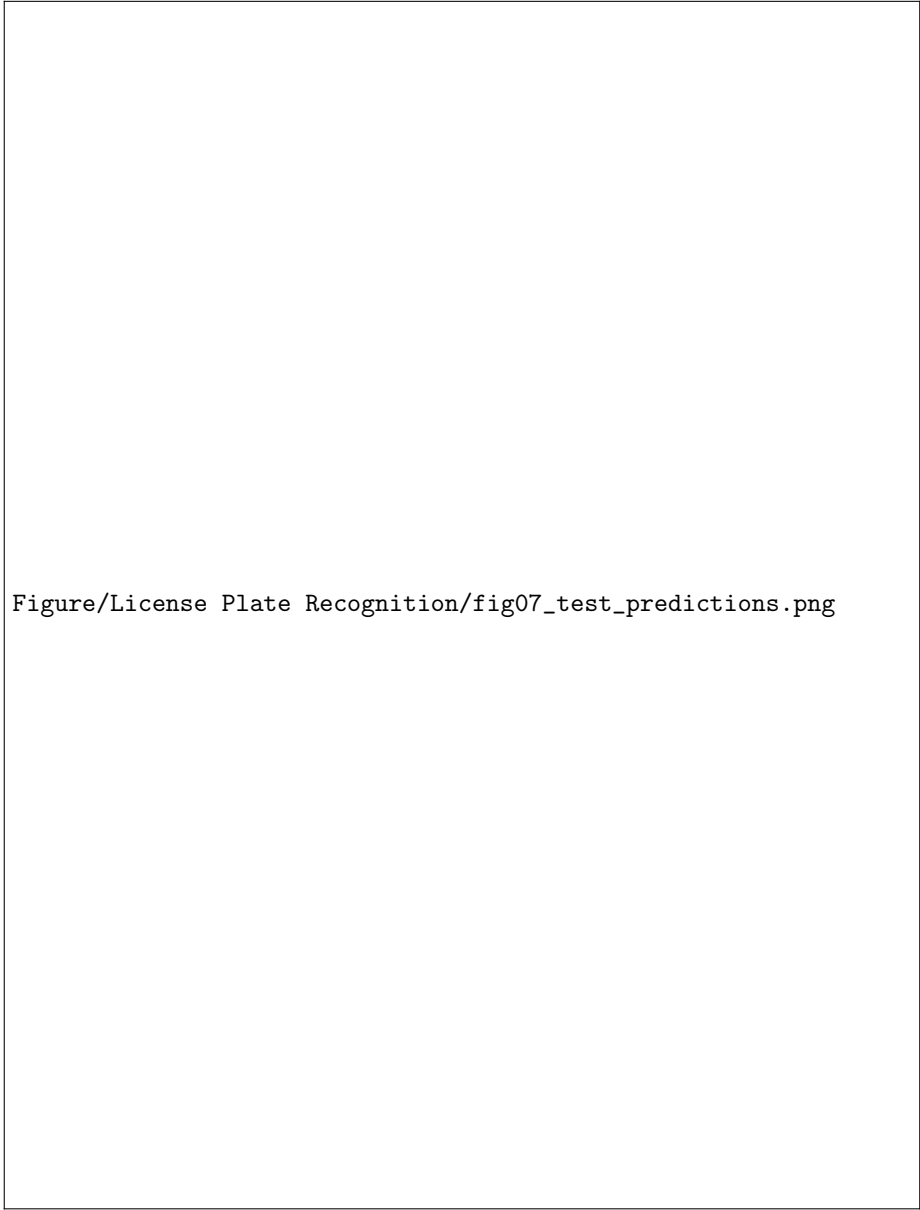
4.4.4 Test set evaluation

The detection metrics obtained on **1,164 test images** partitioned at the base-stem level (with no contamination between splits) are reported in Table 8.

Table 8: Detection metrics on the test set.

Metric	Value
mAP@50	0.9714
mAP@50-95	0.6680
Precision	0.9704
Recall	0.9484

Inference time (CPU, i5-1135G7): preprocessing 1.1 ms, inference 64.6 ms, postprocessing 0.8 ms — approximately 66.5 ms per image (~ 15 FPS). Precision 0.97 and recall 0.95 indicate a good balance, the lower mAP@50-95 of 0.668 is expected at the stricter overlap thresholds. Qualitative behaviour is illustrated in Figure 20, which shows the model’s predictions on 12 randomly selected test images: bounding boxes are tightly localized across diverse scenes, illumination conditions, and plate orientations, visually corroborating the quantitative metrics above.



Figure/License Plate Recognition/fig07_test_predictions.png

Figure 20: Model predictions on 12 random test set images.

The threshold-dependent behaviour of the detector is summarized by the F1 and Precision–Recall curves in Figure 23: the F1 curve (Figure 23a) attains its maximum at an intermediate confidence threshold, while the Precision–Recall curve (Figure 23b) remains close to the upper-right corner across the operating

range, indicating that high precision is preserved even as recall increases.

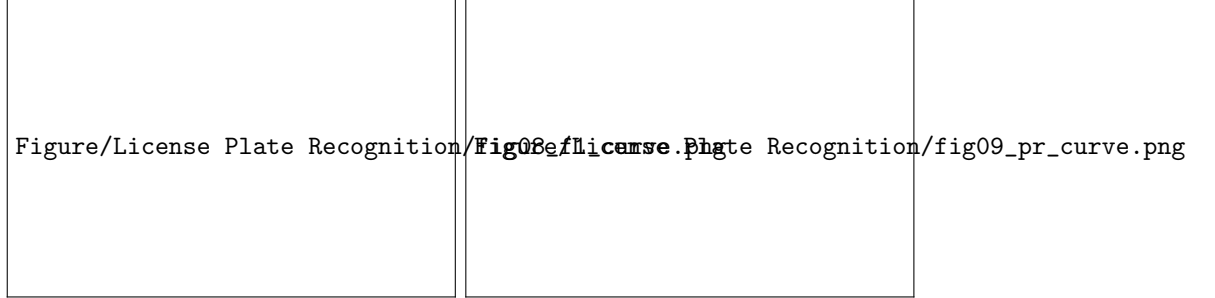


Figure 21: *
(a) F1-score curve

Figure 22: *
(b) Precision-Recall curve

Figure 23: F1 and Precision-Recall curves on the validation set.

4.4.5 Overall statistics

The OCR pipeline was evaluated on 768 Kosovar test images (the 396 Kaggle test images were excluded since they have no RKS regional code). The aggregate statistics, reported in Table 9, show that YOLO detected a license plate in every image (768/768), whereas EasyOCR yielded a non-empty output in 433 cases (56.4% coverage), with 83.83% accuracy on those readings.

Table 9: OCR statistics on the test set.

Metric	Value
Total Kosovo images	768
YOLO detections	768 (100%)
Empty OCR output	335
OCR with output (coverage)	433/768 = 56.4%
Accuracy when OCR yielded output	83.83%

4.4.6 Confusion matrix and per-region metrics

The classification behaviour of the OCR-extracted region is summarized by the 6×6 confusion matrix in Figure 24, which reveals the inter-regional misclassifications more frequently encountered by the pipeline. The corresponding precision, recall, and F1 scores per region are reported in Table 10 and visualized as a grouped bar chart in Figure 25, allowing direct comparison across the six regions.



Figure 24: 6×6 confusion matrix for the OCR-extracted region.

Table 10: Region-level classification.

Region	Precision	Recall	F1	Support
01 — Prishtina	0.81	0.78	0.79	79
02 — Mitrovica	0.92	0.89	0.90	73
03 — Peja	0.76	0.88	0.81	77
04 — Prizren	0.88	0.87	0.88	61
05 — Ferizaj	0.80	0.77	0.79	79
06 — Gjilan	0.92	0.84	0.88	64
Macro Average	0.85	0.84	0.84	433

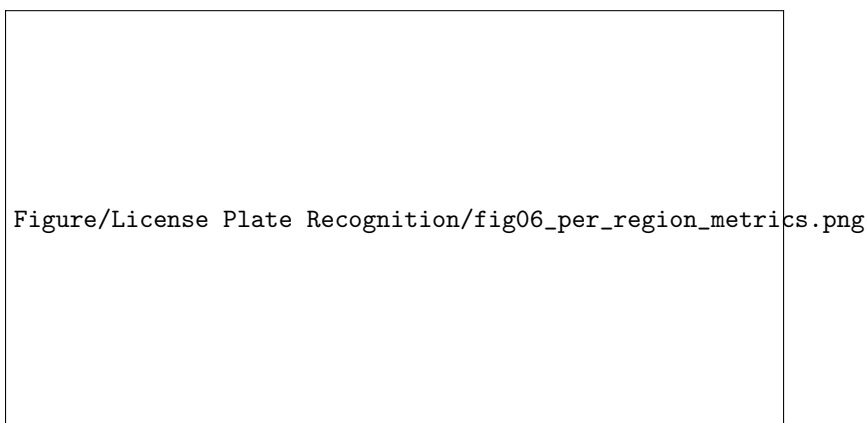


Figure 25: Bar chart of precision/recall/F1 metrics across the six regions.

As summarized by Table 10 and Figure 25, the highest performance is observed for:

- **02 — Mitrovica** (F1=0.90) — the formal contrast of “0/2”.

- **06 — Gjilan** (F1=0.88) — similar behaviour.
- **04 — Prizren** (F1=0.88) — consistent.

The lowest performance, in turn, concerns:

- **01 — Prishtina** (F1=0.79) — “01” is confused with “07” or “OI”.
- **05 — Ferizaj** (F1=0.79) — “05” $\rightarrow$ “06”/“08” under blur.
- **03 — Peja** (F1=0.81) — “03” $\rightarrow$ “08”/“09”.

The off-diagonal mass in Figure 24 is concentrated precisely in these confusion pairs, in agreement with the visual similarity of the corresponding digit pairs under degradation.

4.4.7 Discussion: 56% OCR coverage

This figure reflects a methodological asymmetry: the test set contains predominantly heavily augmented images ($\pm 12^\circ$ rotation, motion blur, perspective distortion, shadows). YOLO retains high robustness (mAP@50= 0.971), but Easy-OCR, pre-trained on clean text, struggles with pronounced degradations. This trade-off is justifiable — strong augmentation is necessary for generalization to real images, and as Section 4.5 demonstrates, coverage rises to 100% on natural images.

4.4.8 Qualitative evaluation on 12 test set samples

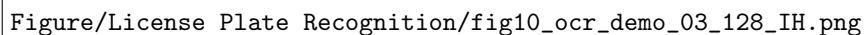
To complement the aggregate statistics with a sample-level perspective, twelve test images were inspected manually, the per-image outcomes are listed in Table 11, which records the YOLO confidence, the OCR-decoded plate, and a binary success indicator.

Table 11: OCR demo on 12 test set images.

#	Filename	Region	YOLO Conf	Read plate	Success
1	02_M-25_aug14	02	49%	RKS ??-343-EJ	No
2	01_PR-02_aug14	01	68%	RKS 01-768-HB	Yes
3	05_F-05_aug3	05	55%	RKS 05-272-IL	Yes
4	04_PZ-08_aug14	04	85%	RKS 04-886-JC	Yes
5	03_P-14_aug4	03	—	No detection	No
6	04_PZ-08_aug5	04	47%	RKS 04-886-JC	Yes
7	03_P-05_aug11	03	56%	RKS 03-128-IH	Yes
8	06_G-17_aug2	06	95%	RKS ??-100-QO	No
9	01_PR-07_aug7	01	65%	RKS 01-334-PR	Yes
10	05_F-36_aug7	05	52%	RKS 03-018-ZO	Wrong region
11	06_G-21_aug2	06	91%	RKS 06-308-FM	Yes
12	02_M-19_aug9	02	74%	RKS ??-123-SO	No

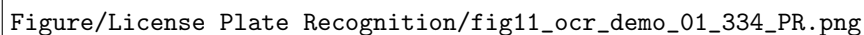
The aggregate over Table 11 is: detection 11/12 (91.7%), region correct 7/12 (58.3%), region “??” 3/12, wrong region 1/12, no detection 1/12. Three representative cases are examined in detail to illustrate the spectrum of pipeline

behaviour. Figure 26 shows the successful decoding of plate “03-128-IH” from more than twenty fragmentary OCR hypotheses, where the weighted-voting mechanism converged on the correct middle-digit sequence despite substantial noise. Figure 27 reports a similarly successful case (“01-334-PR”) in which several individual fragments were corrupted, yet the cascade reconstructed the full plate through the dual-allowlist and prefix-penalty logic. By contrast, Figure 28 documents a failure mode in which a Ferizaj plate (region 05) was misread as region “03”, consistent with the “05” $\rightarrow$ “03” confusion identified in Figure 24.



Figure/License Plate Recognition/fig10_ocr_demo_03_128_IH.png

Figure 26: OCR demo “03-128-IH” — success with 20+ fragments.



Figure/License Plate Recognition/fig11_ocr_demo_01_334_PR.png

Figure 27: OCR demo “01-334-PR” — success despite corrupted fragments.



Figure/License Plate Recognition/fig12_ocr_demo_failure_case.png

Figure 28: OCR demo failure — Ferizaj (05) read as “03”.

4.5 Real-World Evaluation (Deployment)

Evaluation on real images, drawn independently from the training distribution, constitutes the most meaningful test of generalization.

4.5.1 Experimental protocol

A set of ten photographs was acquired with a mobile-phone camera under typical conditions: 1–3 m distance, natural daylight, predominantly frontal viewpoints with only mild deviations, and good image quality with no induced motion blur. All images were processed end-to-end through the same YOLO + OCR pipeline used for the test-set evaluation, with no parameter retuning. The samples were chosen to cover all six regions, ensuring that the deployment evaluation is not biased towards any specific regional code.

4.5.2 Full results

The per-image outcomes of the deployment evaluation are reported in Table 12, which lists the inferred region, the YOLO confidence, the OCR-decoded plate, and a binary success indicator for each photograph. The aggregate metrics derived from Table 12 are summarized in Table 13, a graphical synthesis is provided in Figure 29, which combines per-image confidences, correctness flags, and regional coverage in a single view.

Table 12: Results on 10 real-world photographs.

#	Photo	Region	YOLO Conf	Read plate	Success
1	Foto1	06 — Gjilan	38.5%	RKS 06-880-CN	Yes
2	Foto2	06 — Gjilan	57.7%	RKS 06-779-HE	Yes
3	Foto3	02 — Mitrovica	68.5%	RKS 02-198-GS	Yes
4	Foto4	02 — Mitrovica	87.1%	RKS 02-205-FN	Yes
5	Foto5	01 — Prishtina	61.7%	RKS 01-959-OM	Yes
6	Foto6	01 — Prishtina	55.9%	RKS 01-970-EZ	Yes
7	Foto7	05 — Ferizaj	84.3%	RKS 05-242-IK	Yes
8	Foto8	03 — Peja	39.2%	RKS 03-525-JR	Yes
9	Foto9	04 — Prizren	39.2%	RKS 04-506-IT	Yes
10	Foto10	04 — Prizren	33.7%	RKS 04-505-LF	Yes

Table 13: Real-world performance.

Metric	Value
Detection rate	10/10 = 100%
Full OCR read	10/10 = 100%
Region accuracy	10/10 = 100%
YOLO confidence (range)	33.7%–87.1%



Figure/License Plate Recognition/fig13_deployment_dashboard.png

Figure 29: Graphical summary of the deployment evaluation.

As shown in Table 13 and Figure 29, the system attained perfect performance on all three axes — detection, full OCR read, and regional classification — with YOLO confidences spanning a broad range (33.7%–87.1%), suggesting that the pipeline remains accurate even when the detector itself is uncertain.

4.5.3 Illustrative examples

To illustrate the operating regimes spanned by the deployment set, three representative cases are presented in Figures 30–32. Figure 30 shows a high-

confidence detection (87%) in which the plate is clearly visible in the foreground, Figure 31 corresponds to a medium-confidence detection (62%) in which the voting mechanism correctly selected “01” despite the presence of two candidate regions in the OCR fragments, and Figure 32 reports a low-confidence detection (34%) for which the reading remained correct, confirming the functional robustness of the pipeline at the lower end of the confidence spectrum.

Figure/License Plate Recognition/fig14_real_high_conf_87pct.png

Figure 30: High-confidence detection (87%) — RKS 02-205-FN.

Figure/License Plate Recognition/fig15_real_medium_conf_2candidates.png

Figure 31: Medium-confidence detection (62%) — RKS 01-959-OM.

Figure/License Plate Recognition/fig16_real_low_conf_34pct.png

Figure 32: Low-confidence detection (34%) — RKS 04-505-LF.

4.5.4 Comparison with the test set

A side-by-side comparison between the augmented test set and the real-image set is presented in Table 14, which contrasts detection rate, OCR coverage, and regional accuracy across the two evaluation regimes.

Table 14: Performance comparison: test set vs real-world images.

Aspect	Test set (1164 augm.)	Real images (10)
Detection rate	100% (768/768 Kosovo)	100% (10/10)
OCR coverage	56.4%	100%
Region accuracy	83.83%	100%

The discrepancy revealed by Table 14 — particularly the jump in OCR coverage from 56.4% to 100% — is consistent with the nature of the two distributions: the test set contains heavy synthetic transformations that do not occur in natural photographs. The model detects robustly even under degradation, while the OCR operates with high success on clean text. Notably, YOLO confidences vary widely on real images (33.7%–87.1%) yet OCR remains at 100% — low-confidence boxes still correspond to good localizations, which makes the pipeline functionally robust.

4.5.5 Analysis of three cases

To clarify how the voting mechanism behaves in practice, three of the cases listed in Table 12 are dissected at the fragment level.

Photo 5 (Prishtina): the fragments [95904, 0, 01959, 04, 01, 95908, 059, ...] contained two candidate regions, the voting mechanism selected “01” (2 occur-

rences vs 1 for “04”). The mechanism behaved as expected. The final reading is shown in Figure 31.

Photo 7 (Ferizaj): the fragments [24210, 05, 2421, 055, 242416, 052428, ...] contained two longer fragments (seven and six digits) from which the three middle digits “242” can be extracted either as a prefix of “242416” or after stripping the regional code from “052428”. Both interpretations converged, confirming the utility of the prefix logic.

Photo 9 (Prizren): the fragments [50611, 506, 04, ...] contained the spurious candidate “506”, the regional-prefix penalty (Listing 3) combined with positional weighting neutralized this risk. The penalty proved essential under such ambiguity, as illustrated by the low-confidence successful reading in Figure 32.

4.6 Discussion

Hyperparameters and bias–variance positioning. The v8n vs v8s comparison confirmed a measurable effect: the smaller model achieved $\text{mAP@50}=0.9477$ versus 0.9419, directly illustrating the bias–variance tradeoff in the limited-data regime. The increase in resolution (832×832) was planned but not completed. A methodological limitation is the end-to-end nature of YOLOv8 — direct variation of filter sizes, padding, stride, or pooling would require rebuilding the architecture from scratch and would forfeit the benefits of transfer learning from COCO. Experimentation was therefore confined to dimensions that admit a clean comparison, the final results ($\text{mAP@50}=0.971$ on the test set, 100% on real images) justify this conservative choice.

Feature representation. Visual features are learned automatically by YOLO, the CSPDarknet backbone extracts hierarchical representations P1–P5 that are aggregated by the FPN/PAN neck. The methodological decision to use single-class detection + OCR is itself a form of feature engineering at the level of problem formulation: it focuses the model on the invariant structure of a license plate and yields a more transferable representation. Strong augmentation amplifies this effect.

Interpretability. YOLO detectors are less interpretable than monolithic classifiers, but several analytical instruments do offer limited insight. The distribution of training-set bounding boxes, presented in Figure 33, characterizes the typical sizes, aspect ratios, and image positions occupied by the plates — a prior that the network internalizes during training. Qualitative predictions on a validation batch are shown in Figure 34, where the bounding boxes are consistently localized over the plate regions, confirming that the learned representation focuses on the expected structural cues rather than on incidental scene context. Heatmap-based explanations such as Grad-CAM could in principle highlight the most influential spatial regions, but their implementation for a multi-scale Head is non-trivial and was not pursued in this work.



Figure 33: Distribution of bounding boxes in the training set.

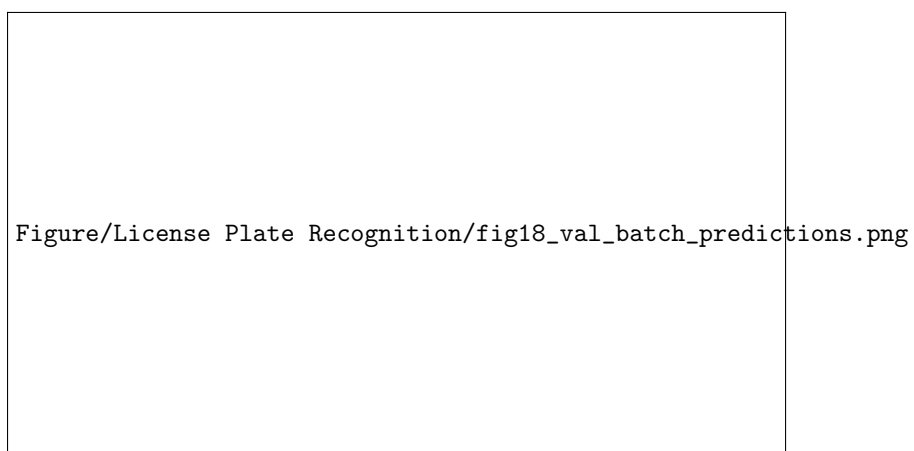


Figure 34: Model predictions over a validation batch.

Iterative refinements — four principal corrections.

1. **Elimination of data leakage:** the initial split distributed augmented files without considering the underlying base image (yielding an unrealistic $\text{mAP@50} \sim 0.99$), grouping by base stem (Listing 1) produced an unbiased estimate of 0.971.
2. **Region-logic correction:** direct reading of the region from the corresponding structure (rather than from the field of middle digits).
3. **Bounding-box deduplication:** the standard NMS ($\text{IoU} = 0.45$) allowed duplicates, an additional deduplication step at $\text{IoU} = 0.5$ was added (Listing 2) together with a lowered NMS parameter of 0.3.
4. **Weighted voting:** fragments with a regional prefix (e.g. “0398”) generated spurious candidates (“039”) that dominated the vote, a scheme was introduced with a $0.25\times$ penalty for candidates beginning with the regional code and a $1.4\times$ boost for fragments in the central zone (Listing 3).

Real-image accuracy rose from $\sim 50\%$ (initial version) to 100% (final version).

Positioning in the literature. Contemporary studies report mAP@50 in the 90–98% range on large datasets ($> 10,000$ images). Our result (0.971 on 1,164 images, free of data leakage) is competitive given (i) the modest dataset size ($\sim 6,900$ after augmentation), (ii) the geographic specificity of Kosovar plates, which is uncovered by existing benchmarks, and (iii) initialization from COCO without domain knowledge of license plates. The 100% figure on the 10 real images should be interpreted with caution — on a larger test set we would expect a stable value in the 90–100% range.

4.7 Conclusion

The study implemented and evaluated a complete ALPR system for Kosovar plates:

1. **Detection:** $\text{mAP@50} = 0.9714$ over 1,164 test images free of data leakage.
2. **OCR + region:** 83.83% on the test set, 100% on real mobile-phone images.
3. **Real-world:** 10/10 images detected and read correctly, covering all 6 regions.

Research questions:

- (i) v8n (3.0M) outperformed v8s (11.2M) — the bias–variance tradeoff in action.
- (ii) features are learned automatically, the single-class + OCR formulation constitutes feature engineering at the level of problem definition.

- (iii) the bounding-box distribution and qualitative predictions evidence the learning of an invariant representation.

Methodological contributions: identification and correction of the data-leakage flaw, the single-class + OCR design choice, an OCR pipeline with 6 preprocessing variants, dual-allowlist decoding, and weighted voting, four documented iterative refinements, and a modular, reproducible structure (**step0** through **step6**).

Limitations: the 832×832 experiment was not completed, EasyOCR is sensitive to augmentation (56% coverage on the test set, a justifiable trade-off), the Kosovar source is modest in size (270 images), statistical power on the real-image set is limited ($n = 10$), and only the six standard regions are covered.

Future work: a dedicated OCR for Kosovar plates, expansion of the dataset to thousands of original images per region, completion of **exp3** and the testing of YOLOv8m/l, YOLOv9, and YOLOv10, real-time deployment via ONNX, TensorRT, CoreML, or TFLite, and functional extension to the detection of traffic signs.

The study shows that combining a contemporary detector (YOLOv8) with a carefully calibrated OCR pipeline can achieve high performance on a real ALPR task even with a modest dataset. The principal success factors were a careful problem formulation (single-class detection + OCR), methodological rigor (the elimination of data leakage), and iterative refinement guided by empirical diagnostics. The 100% accuracy on real images, while limited in statistical power, constitutes an encouraging indicator of practical robustness.

GitHub Repositories

The source code for the two main components of this project is available on GitHub:

- **IRIS CNN Classification:**
<https://github.com/RigonP/iris-cnn-classification>
- **Kosovo License Plate Detection:**
<https://github.com/RigonP/kosovo-license-plate-detection>

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.

- [2] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.
- [4] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023. GitHub repository.
- [5] JaidevAI. EasyOCR: Ready-to-use OCR with 80+ supported languages. <https://github.com/JaidevAI/EasyOCR>, 2020. GitHub repository.
- [6] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [7] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- [8] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [9] Andrewmvd. Car license plate detection. <https://www.kaggle.com/datasets/andrewmvd/car-plate-detection>, 2020. Kaggle Dataset.